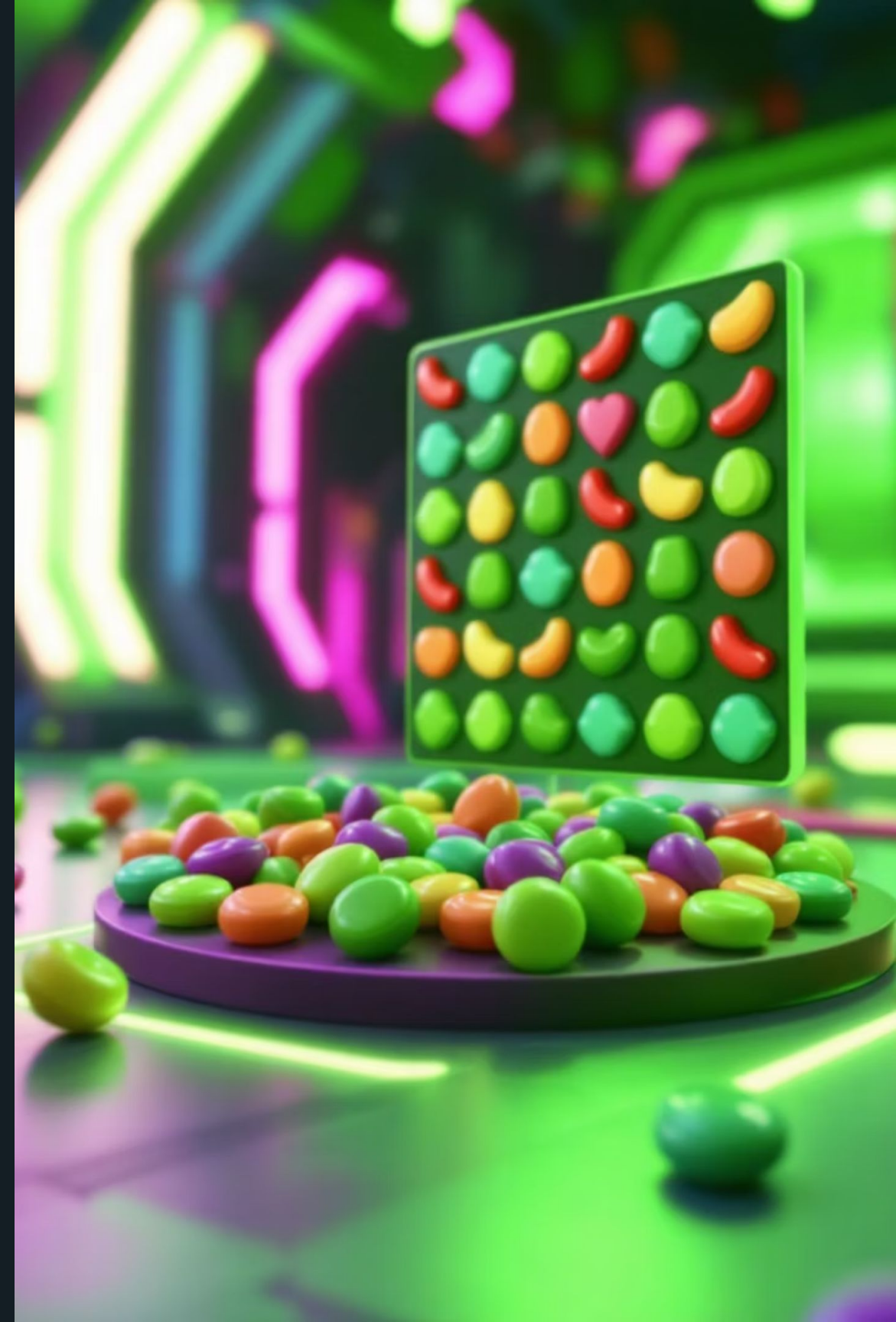


CandyCrush Automator: Aplicație cu interfață grafică și strategie automată

Acest proiect prezintă o aplicație software pentru automatizarea jocului Candy Crush, oferind o interfață grafică pentru vizualizare și o strategie automată de maximizare a scorului. Vom discuta tehnologii, arhitectură, modelarea jocului (stări, tranziții), strategia euristică, rezultate (100 simulări), limitări și integrarea AI.



Scopul Aplicației: De la Modelare la Maximizarea Scorerului

Modelarea Jocului Candy Crush

Reprezentarea digitală a jocului: stări, tipuri de bomboane, reguli de combinație. Esențial pentru automatizare.

Automatizarea Mutărilor

Algoritm care identifică și execută mutări autonome, simulând jucătorul. Detectează swap-uri și cascade.

Maximizarea Scorerului

Strategie inteligentă pentru scor maxim, prin combinații speciale și cascade extinse.

Analiză Multi-Jocuri

Colectarea și analiza statisticilor din rulări multiple pentru evaluarea algoritmului.

Tehnologii și Arhitectură Modulară

Proiectul utilizează **Python 3**. GUI-ul este construit cu **Tkinter** pentru vizualizarea jocului.

Arhitectura soluției este modulară, cu funcționalități împărțite în componente distincte:

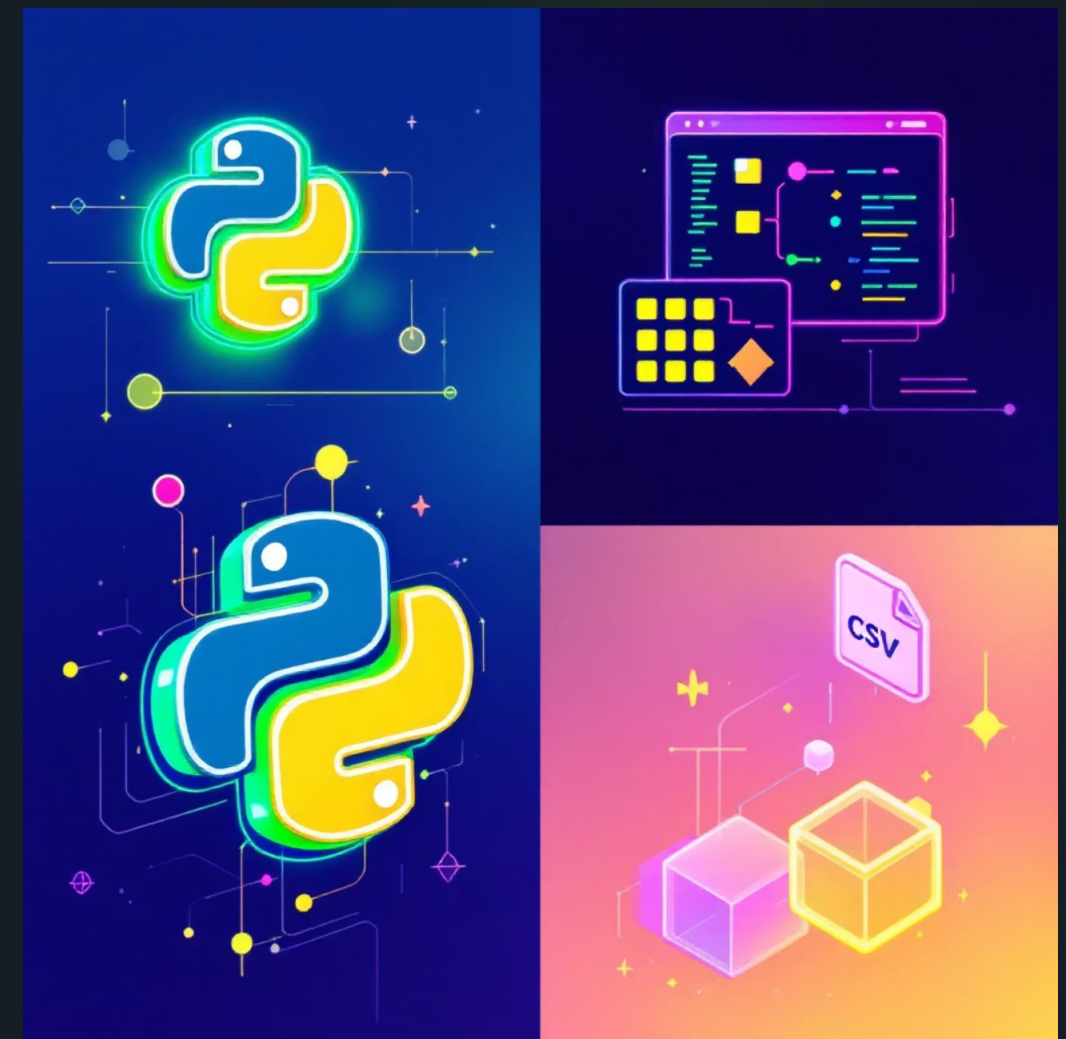
Modul Modelare Joc: gestionează starea, regulile și logica jocului.

Modul Strategie: algoritmi de decizie automată a mutărilor.

Modul Interfață Grafică: afișare vizuală și interacțiuni.

Modul Analiză și Export: înregistrează rezultatele, export **CSV**.

Această abordare facilitează mentenanța, testarea și extinderea proiectului.



Modelarea Jocului: Stări, Tranziții și Detectoare de Șabloane

Jocul Candy Crush este modelat ca o mașină de stări, fiecare stare fiind definită de configurația tablei de joc și parametrii asociați.

Tabla de Joc: Matrice 11x11, unde fiecare celulă conține un tip de bomboană. Dimensiunea asigură un echilibru între complexitate și fidelitate.

- **Starea Completă a Jocului:**

Matricea bomboanelor: Aranjamentul curent al bomboanelor.

Scorul curent: Puncte acumulate.

Nivelul curent: Nivelul jocului.

Tranziții între Stări: O mutare validă implică:

Swap (Schimbare): Interschimbarea a două bomboane adiacente (orizontal/vertical).

Detecție (Detectare): Verificare șabloane de 3+ bomboane identice, adiacente ("match-uri").

Eliminare: Bomboanele "match" sunt eliminate, scorul este actualizat.

Cascade Automate: Bomboanele cad pentru a umple spațiile, iar noi bomboane sunt generate. Acest proces poate declanșa noi "match-uri" și cascade.

Detectoare de Șabloane: Algoritmi care identifică grupuri de 3+ bomboane identice pe orizontală sau verticală, esențiale pentru validarea mutărilor și reacțiile în lanț.

Exemplu de Cod: Mecanica de Swap și Validare

Acest fragment de cod ilustrează logica de bază pentru efectuarea și validarea unei mutări (swap)

în joc, asigurând că doar mutările productive sunt reținute.

```
def try_swap(self, a, b):    self.swap(a, b)    if  
self.detect_formation():    self.cascade()    return  
True    self.swap(a, b)    return False
```

Explicația funcției try_swap:

`self.swap(a, b)`: Interschimbă bomboanele la pozițiile a și b.

`if self.detect_formation()`: Verifică dacă swap-ul a creat șabloane (match-uri).

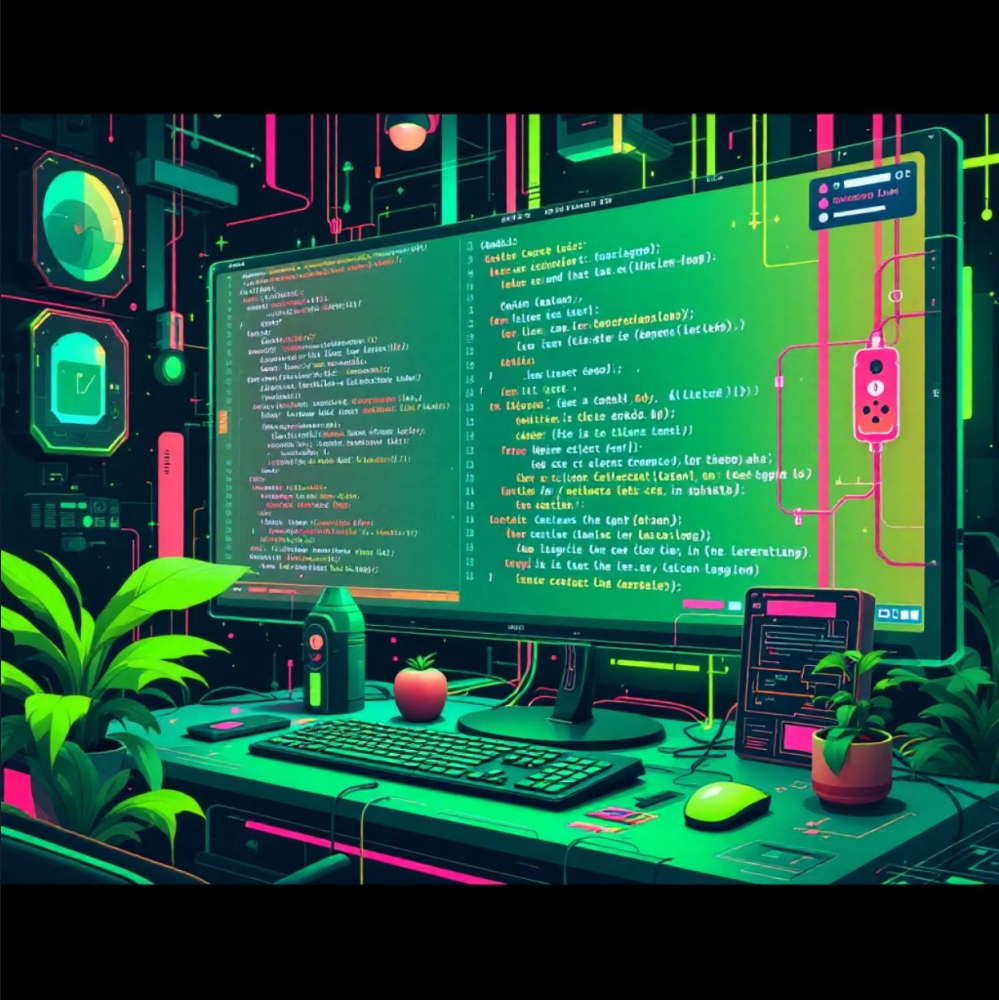
`self.cascade()`: Dacă există match-uri, declanșează cascada (eliminare, cădere, reumplere).

`return True`: Returnează True dacă mutarea este productivă.

`self.swap(a, b)` (a doua oară): Dacă nu există match-uri, anulează swap-ul.

`return False`: Returnează False dacă mutarea este inefficientă.

Acest mecanism acceptă doar mutările care contribuie la progresul jocului.



Exemplu de Cod: Implementarea Mecanismului de Gravitație

Mecanismul de gravitație este crucial în jocurile match-3, asigurând reumplerea tablei după eliminarea bomboanelor. Codul ilustrează o implementare simplificată.

```
for c in range(cols):
    col = [grid[r][c] for r in range(rows) if grid[r][c] != 0]
    col = [0] * (rows - len(col)) + col
    for r in range(rows):
        grid[r][c] = col[r]
```

Explicația funcției de gravitație:

`for c in range(cols):` Parcurge fiecare coloană.

`col = [...]` Creează o listă cu bomboanele non-zero din coloană.

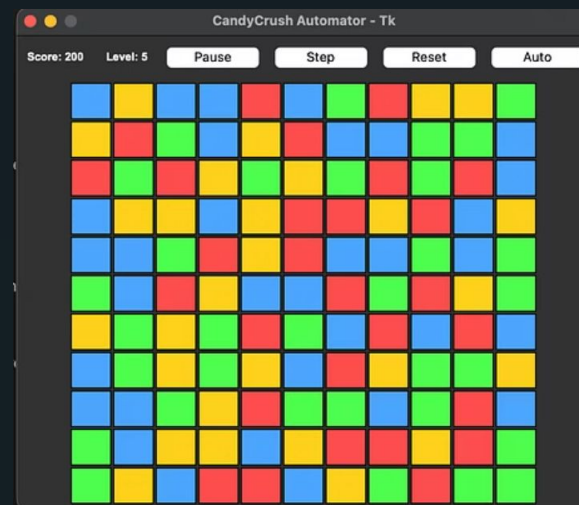
`col = [0] * (rows - len(col)) + col` Reumple coloana cu noi elemente (0) sus și mută bomboanele existente în jos.

`for r in range(rows): grid[r][c] = col[r]` Actualizează tabla de joc cu noua stare a coloanei.

Acest mecanism asigură un comportament realist al tablei de joc, pregătind-o pentru noi match-uri.



Aplicația în Rulare: O Privire Vizuală



Această imagine prezintă aplicația rulând, cu interfața grafică Tkinter. Se observă tabla de joc 11x11, scorul, nivelul și butoanele de control. Bomboanele sunt distincte prin culori/simboluri. Interfața permite vizualizarea operațiunilor și interacțiunea manuală pentru testare.

Interfața Grafică: Puntea dintre Om și Automat



Tabla Colorată 11×11

Reprezentarea vizuală a matricei de bomboane. Culoarele distinctive facilitează identificarea meciurilor.



Afișare Scorr și Nivel

Feedback dinamic: scorul se actualizează după fiecare cascadă, nivelul indică progresul jocului.



Butoane de Control

Pause: Oprește execuția automată.

Step: O singură mutare pentru depanare.

Reset: Reinițializează jocul.

Auto: Activează jocul automat.



Control Manual și Automat

Interfața permite atât intervenția manuală, cât și rularea automată a strategiei implementate.

Strategia Automată: O Abordare Deterministă și Greedy

Strategia automată implementată este **deterministă și greedy**, axată pe găsirea rapidă a mutărilor valide fără analiză pe termen lung.

Auto-play Determinist: Algoritmul parcurge tabla de joc într-o ordine prestabilită. Identifică și execută primul swap valid găsit, trecând la următoarea stare. Mutările sunt predictibile.

Căutarea Primului Swap Valid: Algoritmul iterează prin celule, încercând swap-uri adiacente. Funcția ``try_swap`` verifică validitatea. Primul swap valid găsit este executat imediat.

Strategie Greedy: Alege prima opțiune validă, fără a considera impactul pe termen lung (ex: un match de 3 în loc de unul mai mare). Avantajul este simplitatea și viteza, dezavantajul este că nu garantează scorul maxim.

Complexitate Redusă: Datorită naturii deterministe și greedy, complexitatea computațională este scăzută. Timpul de calcul minim permite simulări rapide, fiind un punct de plecare pentru algoritmi mai avansați.



Rezultate Experimentale: Analiza Performanței

100 de Jocuri Rulate

Algoritmul a rulat 100 de jocuri distincte, din configurații aleatoare, pentru statistici relevante.

Date Exportate în CSV

Metrice cheie (scor, mutări, timp, cascade) exportate în CSV pentru analiză ușoară.

Analiză Globală

Datele CSV permit calculul mediilor, distribuțiilor și vizualizări grafice.

Limitări și Direcții Viitoare:

Evaluare Mai Adâncă: Implementare algoritmi de căutare (Minimax) sau funcții euristice sofisticate.

Învățare Automată: Utilizarea Reinforcement Learning (RL) pentru o politică optimă de mutări.

Bomboane Speciale: Logici specifice pentru crearea și activarea strategică a bomboanelor speciale.

Analiză Multi-Obiectiv: Includerea altor obiective, pe lângă maximizarea scorului.